# T3 - Memoria

## SISTEMAS OPERATIVOS
## GRADO EN INGENIERÍA INFORMÁTICA

Autores: Profesores de Sistemas Operativos del
Departamento de Arquitectura de Computadores
(UPC- BarcelonaTech)

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

FIB

Grado en Ingeniería Informática

# Licencia



CC BY-NC-SA 4.0

## Atribución/Reconocimiento-NoComercial-CompartirIgual 4.0 Internacional

### Deed

«By Universitat Politècnica de Catalunya - BarcelonaTech (UPC), Any 2025»

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.es

# Índice

- Conceptos básicos
  - Generación del ejecutable y carga
  - Multiprogramación
  - Tipos de direcciones
  - Soporte HW a la gestión de  memoria
  - Tareas del SO
- Servicios básicos para la gestión de memoria
  - Carga de un programa
  - Asignación de memoria
  - Memoria dinámica
  - Compartición de memoria
- Servicios para la optimización del uso de memoria física
  - Carga bajo demanda
  - Librerías dinámicas
  - Copy on write (COW)
  - Memoria virtual
  - Prefetch
- Linux sobre Intel



Generación del ejecutable y carga
Multiprogramación
Tipos de direcciones
Soporte HW
Tareas del SO

# CONCEPTOS BÁSICOS

# Generación del ejecutable y carga (I)

- Para ejecutar un programa
  - Las instrucciones tienen que estar cargadas en memoria
    - Registro Program Counter contiene la dirección de la instrucción que toca ejecutar
    - Instrucciones de salto o llamadas a función contienen la dirección de la instrucción destino
  - Los datos tienen que estar en memoria o en registros
    - Las instrucciones que usan datos de entrada se pueden referir a registros o a posiciones de memoria



# Generación del ejecutable y carga (II)

- Pasos
  - Escribir código en lenguaje de alto nivel
  - **Generar ejecutable en disco** (**compilador y linker**) → genera un fichero
    - Convertir lenguaje de alto nivel en código máquina
    - Asignar direcciones de memoria a instrucciones y datos
  - **Cargar en memoria** el ejecutable (**SO**)
    - Leer de disco el ejecutable, escribirlo en memoria, y hacer que el registro Program Counter apunte a la primera instrucción a ejecutar (punto de entrada del programa)

# Generación de ejecutable y carga (III)

```
/*test.c*/
int i = 0;
void mifuncion() {
}
main(int argc, char *argv[]) {
    i = 5;
    mifuncion();
}
```

gcc -fcf-protection=none -O0 -o test test.c

nm test

```
(...)
0000000000004014 B i
0000000000001130 T main
0000000000001129 T mifuncion
(...)
```

objdump –d test

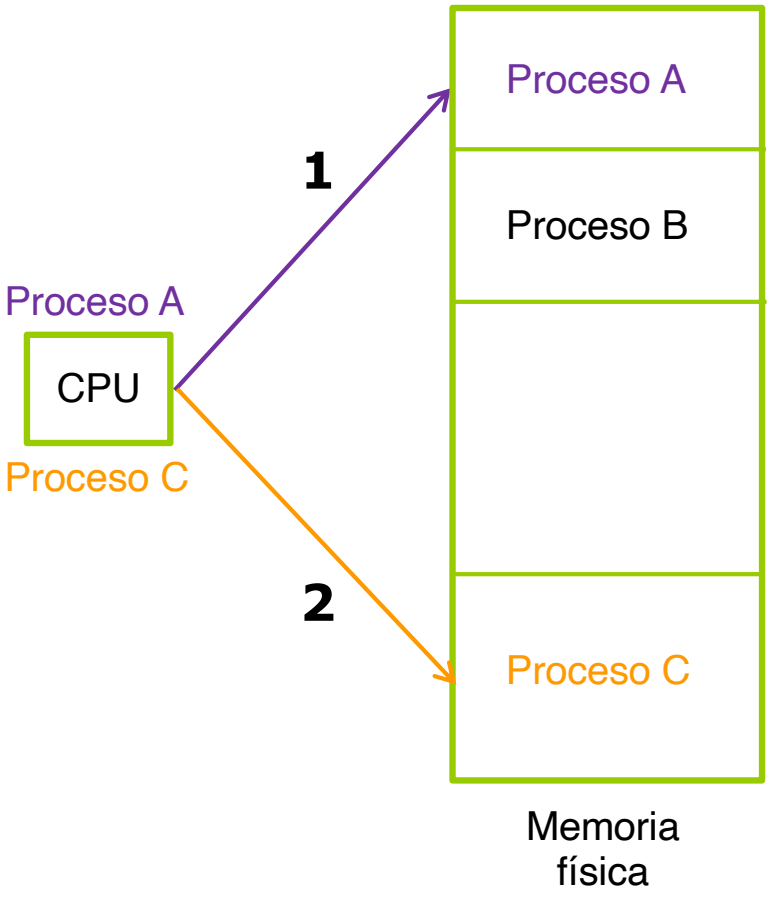
```
0000000000001129 <mifuncion>:
   1129:    55                       push   %rbp
   112a:    48 89 e5                 mov    %rsp,%rbp
   112d:    90                       nop
   112e:    5d                       pop    %rbp
   112f:    c3                       ret

0000000000001130 <main>:
   1130:    55                       push   %rbp
   1131:    48 89 e5                 mov    %rsp,%rbp
   1134:    48 83 ec 10              sub    $0x10,%rsp
   1138:    89 7d fc                 mov    %edi,-0x4(%rbp)
   113b:    48 89 75 f0              mov    %rsi,-0x10(%rbp)
   113f:    c7 05 cb 2e 00 00 05     movl   $0x5, 0x2ecb(%rip) # 4014 <i>
   1146:    00 00 00
   1149:    b8 00 00 00 00           mov    $0x0,%eax
   114e:    e8 d6 ff ff ff           call   1129 <mifuncion>
   1153:    b8 00 00 00 00           mov    $0x0,%eax
   1158:    c9                       leave
   1159:    c3                       ret
```

2ecb+1149=4014

# Multiprogramación (I)

- Sistemas **multiprogramados:** los que permiten tener varios programas cargados a la vez en memoria
  - Facilita la concurrencia: en cada cambio de contexto no hace falta cargar de nuevo el programa



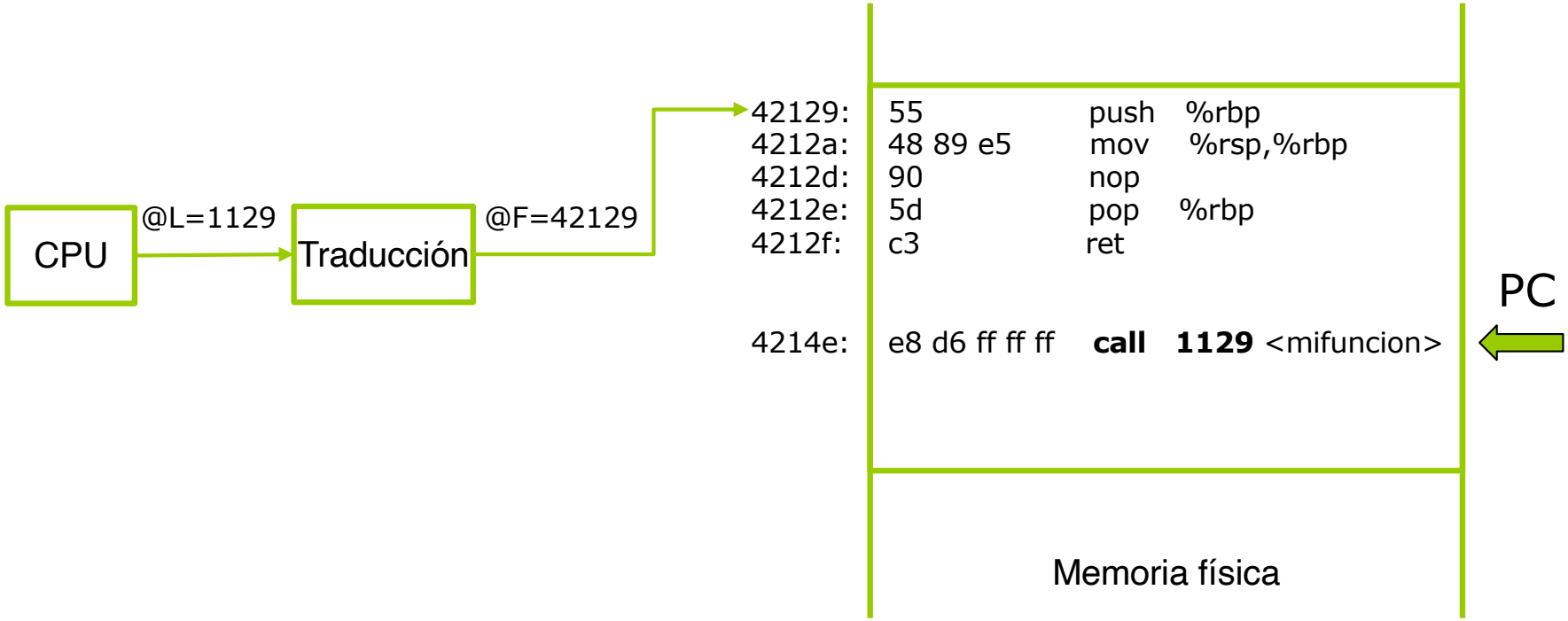

**1**-Proceso A se está ejecutando
**2**-Cambio de contexto a C

# Multiprogramación(II)

- Complica la gestión de memoria implementada por el SO
  - **Protección** de la memoria entre procesos
    - Un proceso solo debe poder acceder a su memoria
  - **Asignación** de memoria a procesos
    - Cuando se pone en ejecución un proceso el SO debe comprobar qué memoria hay disponible (puede ser diferente cada vez)
  - **Traducción** de direcciones
    - Al generar el ejecutable no se puede saber qué direcciones ocupará el programa en cada ejecución
    - Al compilar, se asignan valores que **representan posiciones** en memoria: **direcciones lógicas**
    - Cuando se sabe qué **posiciones reales** ocupa se traduce: **direcciones físicas**
      - En tiempo de carga? → reescribir código y datos para hacer referencia a las direcciones asignadas
        - Resta flexibilidad a las políticas de gestión
      - **En Tiempo de ejecución**

# Tipos de direcciones

- @Lógicas: las que referencia la CPU
  - Generadas en la creación del ejecutable (o en la carga si el SO decide cambiarlas)
- @Físicas: las que ocupa el proceso en memoria
  - Generadas en la asignación de memoria



# Espacios de direcciones

- Espacio de direcciones del procesador
  - Conjunto de @ que el procesador es capaz de emitir (depende del bus de direcciones)
- Espacio de **direcciones lógicas de un proceso**
  - Conjunto de @ lógicas que **un proceso** tiene permitido referenciar
  - Lo inicializa el SO en tiempo de carga y puede ir cambiando durante la ejecución
- Espacio de **direcciones físicas de un proceso**
  - Conjunto de @ físicas (las reales) asociadas al espacio de direcciones lógicas del proceso
  - Lo inicializa el SO en tiempo de carga y puede ir cambiando durante la ejecución



# Soporte HW (I)

- La **traducción** @lógica → @física se hace en tiempo de ejecución por HW
- **SO configura el HW**
  - ¿Qué información?
    - Correspondencia @lógica <--> @física
    - Tipo de accesos permitidos para cada @lógica (lectura, escritura…)
  - ¿Cuándo? Siempre que hay cambios
    - En tiempo de carga
    - Cuando se modifica el espacio lógico o el espacio físico del proceso
    - Cuando hay un cambio de contexto
- **El mecanismo de traducción se usa también para garantizar la protección**
  - El SO solo asigna a un proceso @ físicas de ese proceso



# Soporte HW (II)

- HW que se encarga de la traducción: Memory Management Unit (MMU)
  - Dado un acceso a una @lógica lo intenta convertir en un acceso a la @física correspondiente
  - Si no puede → genera una excepción que gestionará el SO (en algunos casos matando al proceso por acceso ilegal)
    - Cuando el tipo de acceso no está permitido
    - Cuando la @lógica no forma parte del espacio lógico del proceso
    - Cuando la @lógica no tiene una @física asociada

# Tareas del SO en la gestión de memoria

- Carga de programas en memoria
- Asignación de memoria
- Memoria dinámica: reservar/liberar memoria en tiempo de ejecución (mediante llamadas a sistema)
- Compartición de memoria entre procesos
- Servicios para la optimización del uso de memoria
  - Carga bajo demanda
  - Librerías dinámicas
  - Copy on write (COW)
  - Memoria virtual



Carga de un programa

Asignación de memoria

Memoria dinámica

Compartición de memoria entre procesos

# SERVICIOS BÁSICOS DEL SO



# Carga de un programa

- Mecanismo que pone en ejecución un programa contenido en un fichero ejecutable
- En los sistemas POSIX la provoca la llamada a sistema **exec**
- Pasos
  1. Interpretar el fichero ejecutable
  2. Definir el espacio lógico de direcciones del proceso
  3. Actualizar el PCB del proceso
  4. Asignar memoria física y configurar la MMU
  5. Leer secciones del programa de disco y escribirlo en la memoria asignada
  6. Cargar en el *Program Counter* la dirección del punto de entrada del programa (primera instrucción a ejecutar)



# Carga: interpretación del ejecutable

- Los ficheros ejecutables tienen que seguir un formato definido
  - Sección: agrupación de datos o instrucciones con las mismas características
  - Cabecera del ejecutable describe las secciones del programa: tipo, tamaño y posición dentro del fichero
    - Comando Linux para ver la cabecera: objdump –h programa
- Existen diferentes formatos de ejecutable
  - ELF (*Executable and Linkable Format)*: es el más extendido en sistemas POSIX
  - Algunas secciones:

| Nombre sección | Contenido |
|---|---|
| **.text** | **Código del programa** |
| **.data** | **Variables globales inicializadas** |
| **.bss** | **Variables globales sin inicializar** |
| **.rodata** | **Datos solo de lectura** (constantes) |
| .debug | información de debug |
| .comment | información de control |
| .init | código de inicialización del proceso (contiene la @ de la 1ª instrucción) |

# Carga: definición del espacio lógico (I)

- Para cada sección del ejecutable el SO tiene que decidir qué @ lógicas asigna
- Región del espacio lógico: conjunto de @lógicas consecutivas con las mismas características
- Tipos de regiones
  - Code (código): tamaño y contenido se lee del ejecutable
  - Data (variables globales): tamaño y contenido se lee del ejecutable (si son variables inicializadas)
  - Stack (variables locales, parámetros y control de ejecución): tamaño variable durante la ejecución. En el ejecutable tenemos el tamaño inicial y crece hacia direcciones bajas. El contenido se genera en tiempo de ejecución
  - Heap (memoria dinámica): tamaño variable durante la ejecución. Memoria dinámica pedida por el programa en tiempo de ejecución (llamada a sistema brk, función de la libc malloc). Crece hacia direcciones altas. El contenido se genera en tiempo de ejecución
  - Memoria dinámica reservada con mmap: regiones entre stack y heap reservadas cuando el usuario utiliza mmap. El contenido se genera en tiempo de ejecución

# Carga: definición del espacio lógico (II)

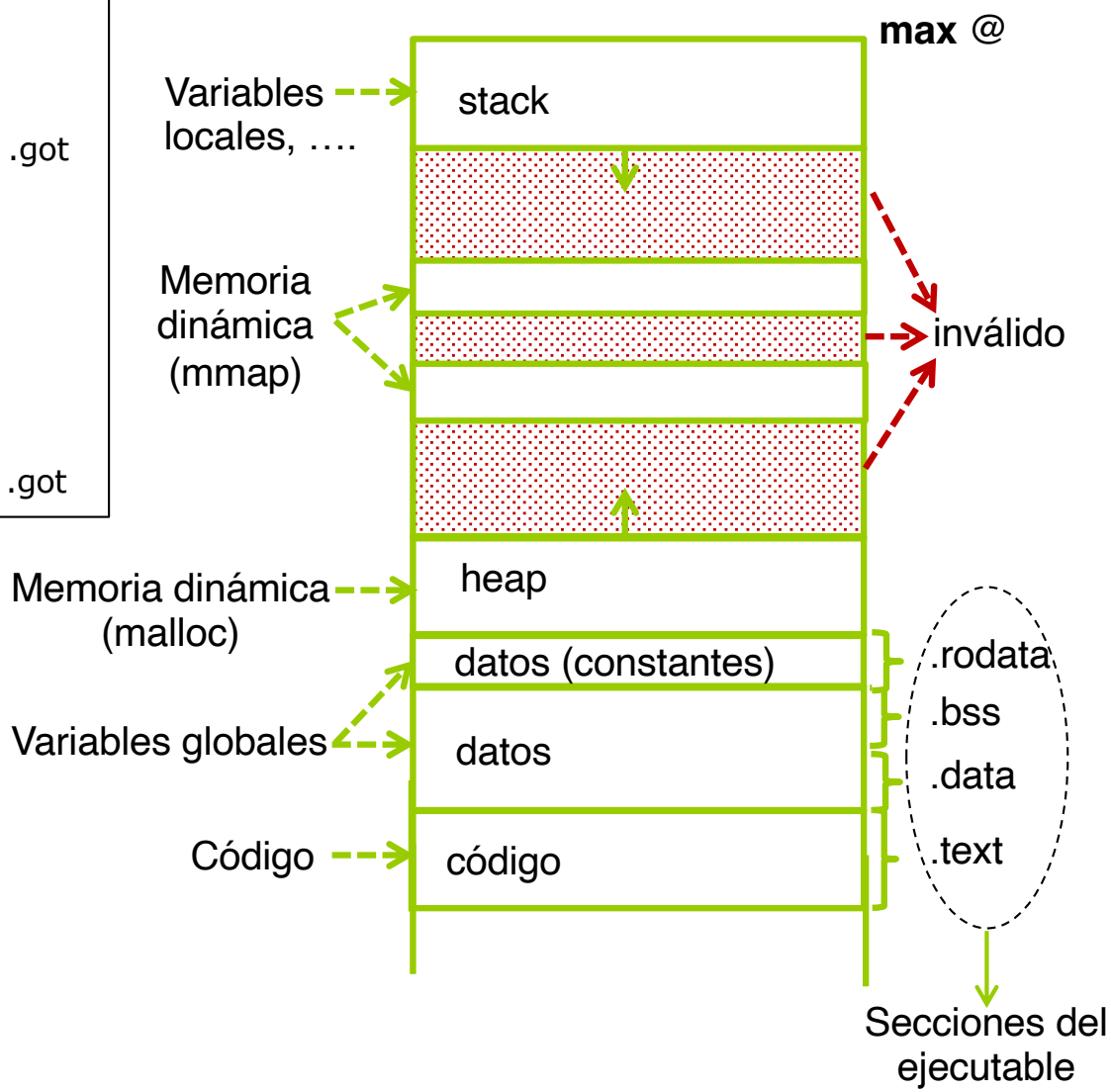
- La dirección 0 suele estar protegida contra acceso para detectar accesos erróneos
- Dirección inicial de cada región
  - La especificada en el fichero ejecutable
  - O el SO le puede aplicar un desplazamiento diferente en tiempo de carga (prevenir ataques como los de inyección de código)
- Una región puede incluir varias secciones del ejecutable
  - Por ejemplo, la región de datos en memoria puede incluir la sección .bss y .data del fichero
- Hay regiones que no se corresponden con ninguna sección del ejecutable (stack, heap, regiones de mmap…)
- El mapeo sección del ejecutable - región de memoria lo decide el linker
  - Puede ser diferente para diferentes versiones de sistema operativo y linker
  - El usuario lo puede modificar en tiempo de linkaje
  - Comando readelf  se puede usar para mostrar el mapeo usado



# Carga: ejemplo de espacio lógico (I)

**readelf –l test**

```
Segment Sections...
  00     .note.gnu.property .note.gnu.build-id
         .note.ABI-tag .rela.plt
  01     .init .plt .text __libc_freeres_fn .fini
  02     .rodata .stapsdt.base .eh_frame
         .gcc_except_table
  03     .tdata .init_array .fini_array .data.rel.ro .got
         .got.plt .data __libc_subfreeres
         __libc_IO_vtables __libc_atexit
         .bss __libc_freeres_ptrs
  04     .note.gnu.property
  05     .note.gnu.build-id .note.ABI-tag
  06     .tdata .tbss
  07     .note.gnu.property
  08
  09     .tdata .init_array .fini_array .data.rel.ro .got
```



# Carga: ejemplo de espacio lógico (II)

- Para cada proceso podemos consultar sus regiones durante la ejecución

@lógica inicial | @lógica final | offset fichero | Origen contenido

**cat /proc/[PID]/maps**

```
1:  00400000-00401000 r--p 00000000 00:2d 10249272                  /home/usuario1/test
2:  00401000-00498000 r-xp 00001000 00:2d 10249272                  /home/usuario1/test
3:  00498000-004c1000 r--p 00098000 00:2d 10249272                  /home/usuario1/test
4:  004c1000-004c5000 r--p 000c0000 00:2d 10249272                  /home/usuario1/test
5:  004c5000-004c8000 rw-p 000c4000 00:2d 10249272                  /home/usuario1/test
6:  004c8000-004cd000 rw-p 00000000 00:00 0
7:  2c27f000-2c2a1000 rw-p 00000000 00:00 0                         [heap]
8:  7ffc1c1bb000-7ffc1c1dc000 rw-p 00000000 00:00 0                 [stack]
9:  7ffc1c1f4000-7ffc1c1f8000 r--p 00000000 00:00 0                 [vvar]
10: 7ffc1c1f8000-7ffc1c1fa000 r-xp 00000000 00:00 0                 [vdso]
11: ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0         [vsyscall]
```

1: Información sobre el formato del ejecutable (cabecera)

**2: Código (.text) –> permiso de ejecución**

**3,4: Constantes (.rodata, metadatos de C y C++) –> solo permiso de lectura**

**5: Variables globales inicializadas y no inicializadas (.data y .bss) → permiso lectura/escritura, con origen en el fichero para leer .data**

6: variables de la libC

**7: Heap**

**8: Stack**

9,10,11: vvar, vdso, vsyscall: para acelerar la ejecución de llamadas a sistema simples y frecuentes (vsyscall es para el mecanismo antiguo)

# Carga: pasos finales

- **Asignación de memoria física**
- Leer del programa ejecutable las secciones y escribirlas en la memoria asignada



# Asignación de memoria: pasos

- Se ejecuta cada vez que un proceso necesita memoria física:
  - En POSIX: creación (fork), carga del ejecutable (exec), petición de memoria en tiempo de ejecución (memoria dinámica), implementación de alguna optimización
- Pasos
  - Buscar memoria física libre y marcarla como ocupada
    - El SO necesita una estructura de datos con esta información
  - Actualizar MMU con la correspondencia entre @lógicas y @físicas
  - Actualizar PCB
    - En el PCB se guarda la descripción de las regiones del proceso y dónde está la información que necesita la MMU
    - Necesario para implementar el cambio de contexto
      - Cuando un proceso vuelve a ocupar la CPU es necesario configurar de nuevo la MMU



# Asignación de memoria: tipos

- ¿Cómo se elige la memoria física?
  - ¿Asignación contigua? Todo el proceso estaría en direcciones físicas contiguas
    - Poco flexible y dificulta aplicar optimizaciones
  - **Asignación no contigua** Visto en EC
    - Basada en
      - **Paginación** (particiones fijas)
      - **Segmentación** (particiones variables)
      - Esquemas combinados
        - Por ejemplo, segmentación paginada

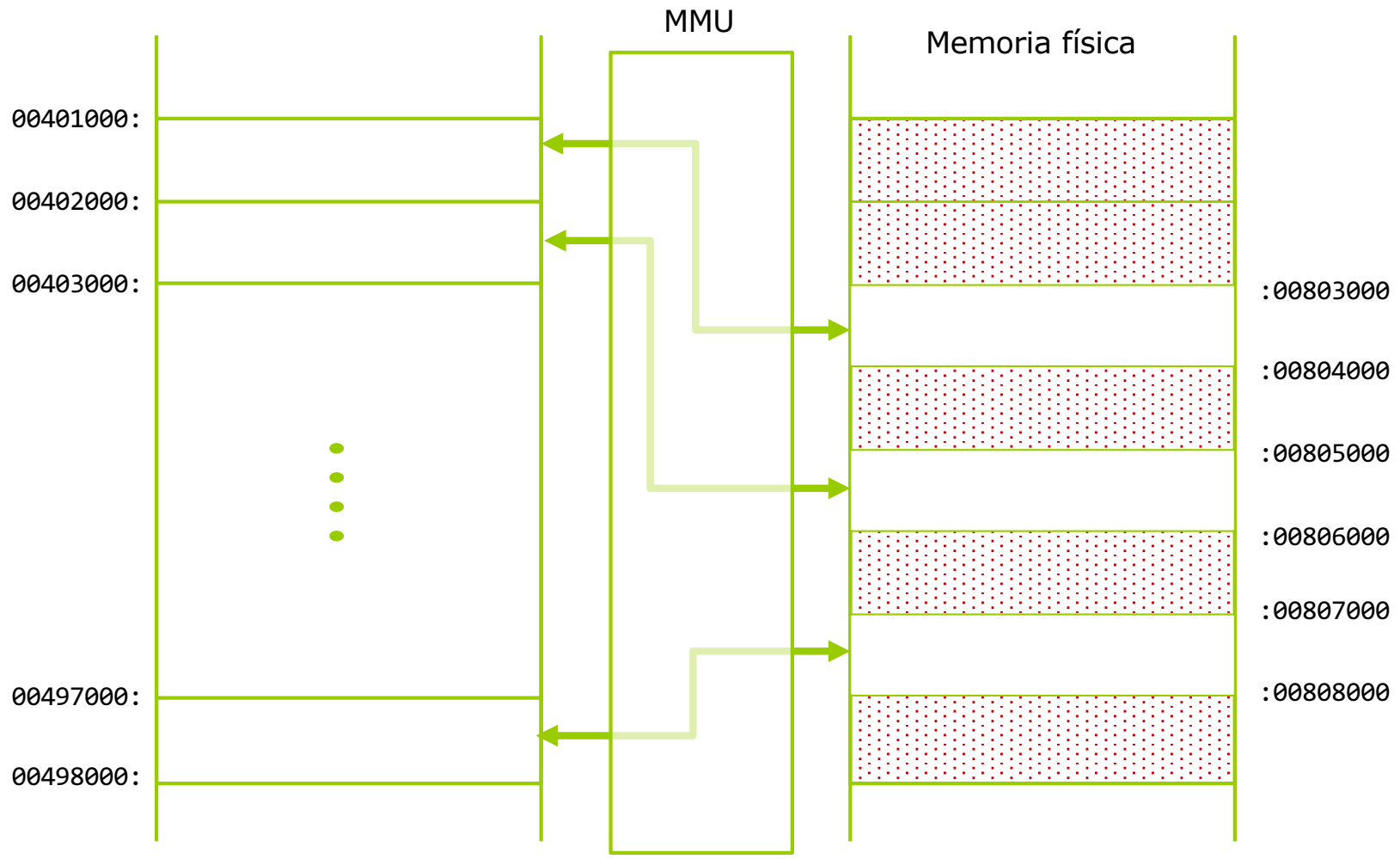
# Asignación de memoria física: Paginación (I)

- Espacio de @lógicas dividido en particiones de tamaño fijo: **páginas**
- Memoria física dividida en particiones del mismo tamaño: **marcos**
- Tamaño de página potencia de 2 (valor habitual 4096 Bytes)
- Asignación
  - Para cada página del proceso buscar un marco libre
    - Lista de marcos libres
- Liberación: cuando un proceso acaba la ejecución (o libera una región de memoria) devolver los marcos asignados a la lista de libres
- **Página: unidad de trabajo del SO**
  - Permite especificar protección a nivel de página
  - Normalmente, por temas de permisos, una página pertenece a una región de memoria (código/datos/heap/pila)
- Presenta fragmentación interna
  - Memoria asignada a un proceso aunque no la va a ocupar
  - Se debe a que el tamaño de una región puede no ser múltiplo del tamaño de una página



# Asignación de memoria física: Paginación (II)

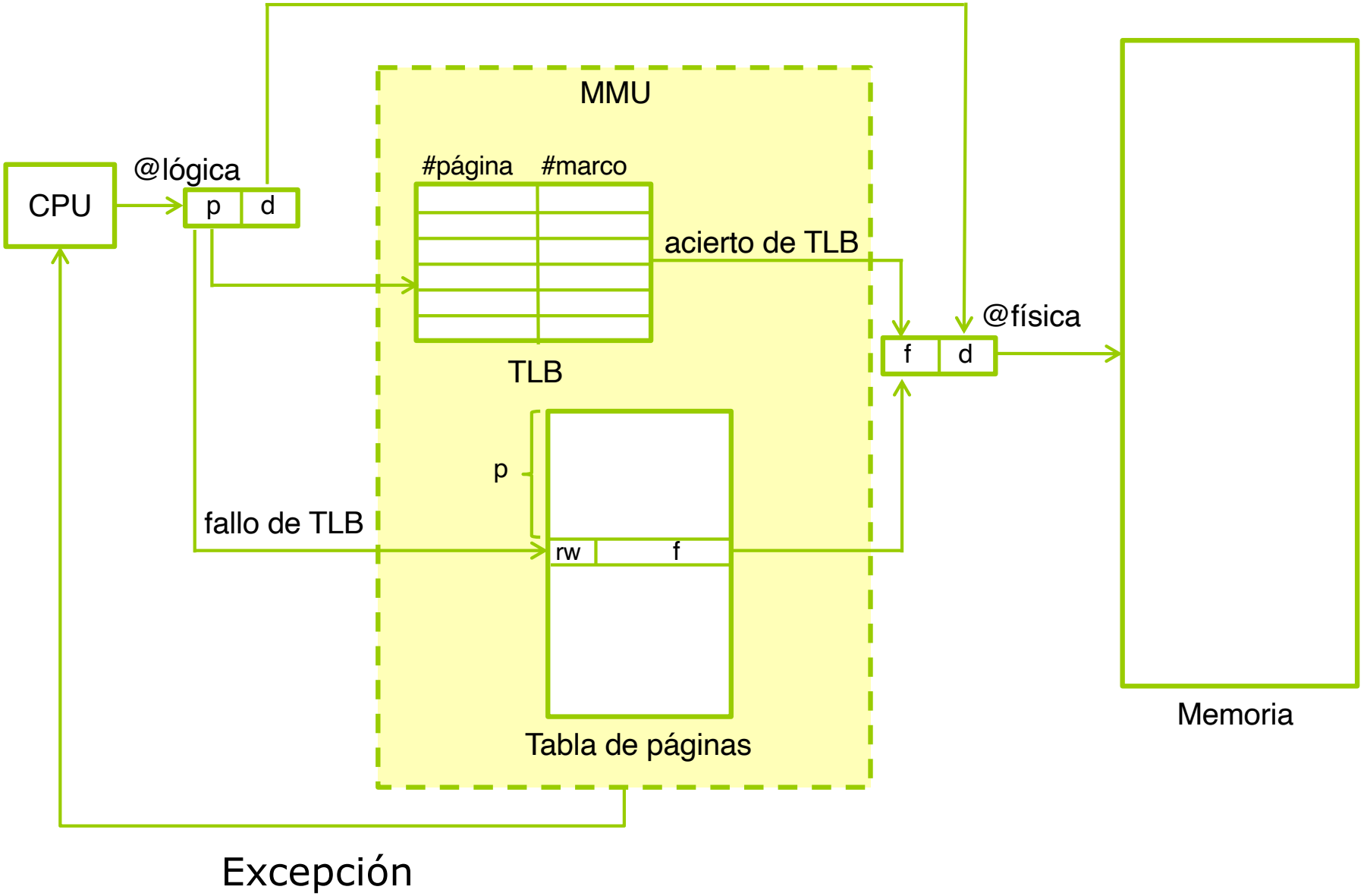
```
00401000-00498000 r-xp 00001000 00:2d 10249272          /home/usuario1/test
```



# Asignación de memoria física: Paginación (III)

- MMU: **Tabla de páginas**
  - Para mantener información a nivel de página: validez, permisos de acceso, marco asociado, etc….
  - Una entrada para cada página
  - Una tabla por proceso
  - Suele guardarse en memoria y SO debe conocer la @base de la tabla de cada proceso (por ejemplo, guardándola en el PCB)
    - Mecanismos para ajustar el tamaño ocupado por cada tabla
  - Procesadores actuales también disponen de TLB (*Translation Lookaside Buffer*)
    - Memoria asociativa (cache) de acceso **más rápido** en la que se almacena la información de traducción para las páginas *activas*

# Asignación de memoria física:Paginación (IV)



# Reservar/Liberar memoria dinámica

- Hay variables cuyo tamaño depende de parámetros de la ejecución
  - Fijar el tamaño en tiempo de compilación no es adecuado
    - O se desaprovecha memoria o se tiene error de ejecución por no haber reservado suficiente
- **Memoria dinámica**: memoria reservada por un proceso durante su ejecución
- SO ofrece llamadas a sistema
  - Para reservar nuevas regiones:
    - Validar nuevas direcciones lógicas
    - Asignar memoria física
    - Actualizar la MMU y el PCB del proceso
  - Para liberar regiones de memoria dinámica no necesarias
    - Invalidar las direcciones lógicas
    - Liberar la memoria física que tenía asignadas
    - Actualizar la MMU y el PCB del proceso

# Linux: memoria dinámica en el heap (I)

- Llamadas a sistema para manipular el heap
  - heap: región dedicada a memoria dinámica
  - brk y sbrk modifican el límite del heap y devuelven el límite anterior al programador

    void * sbrk (int num_bytes)

    - Se pueden usar tanto para aumentar como para disminuir el heap (parámetro negativo)
    - El SO no es consciente de qué variables se están guardando en cada dirección del heap
  - Interfaz poco amigable y en desuso → delegaba la gestión del heap en el código de usuario (cuándo se podía reducir y en qué cantidad).



# Linux: memoria dinámica en el heap (II)

- Funcionalidad implementada por la librería de C para facilitar el uso del heap
  - Implementa funciones para pedir y para liberar memoria consecutiva dentro del heap
  - Intenta reducir el número de llamadas a sistema ejecutadas
    - El heap no tiene por qué reflejar el uso actual que está haciendo el proceso de la memoria
      - La librería puede pedir al SO más memoria de la necesaria y puede retrasar el momento en el que se libera
  - **Estructuras de datos**:
    - Para representar el estado actual del heap
      - Para cada petición de memoria qué direcciones se han asignado
      - Qué direcciones del heap no están en uso



# Linux: memoria dinámica en el heap (III)

- **Función para pedir memoria**

  void *malloc(int num_bytes)

  - Si en el heap **no hay espacio libre consecutivo** suficiente, usa llamadas a sistema (sbrk) para pedir más
    - El tamaño pedido al SO suele ser mayor que el tamaño pedido por el usuario en el malloc: marca como libre la memoria extra
  - Si en el heap **hay espacio libre consecutivo** suficiente, lo marca como reservado en sus estructuras y devuelve la dirección de inicio
  - Si el **tamaño pedido es muy grande** (mayor que una constante definida en el sistema) la **memoria se pide fuera del heap** (usando la llamada a sistema mmap)
- **Función para liberar memoria**

  void free(void * region)

  - Marca como libres las direcciones asociadas a la región (el parámetro tiene que ser una dirección devuelta por un malloc previo)
  - Eventualmente puede usar llamadas a sistema para reducir el tamaño del heap



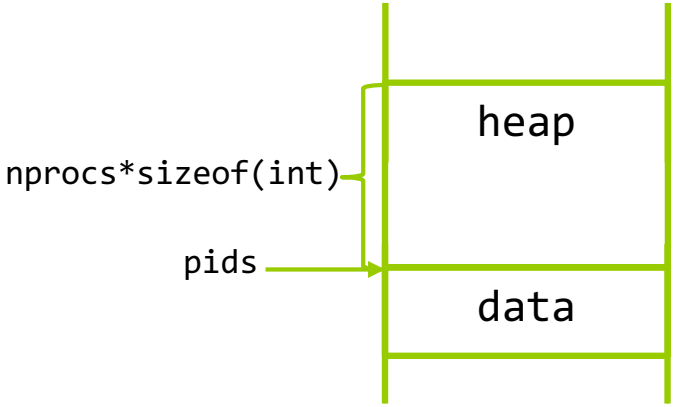
# Linux: memoria dinámica en el heap (IV)

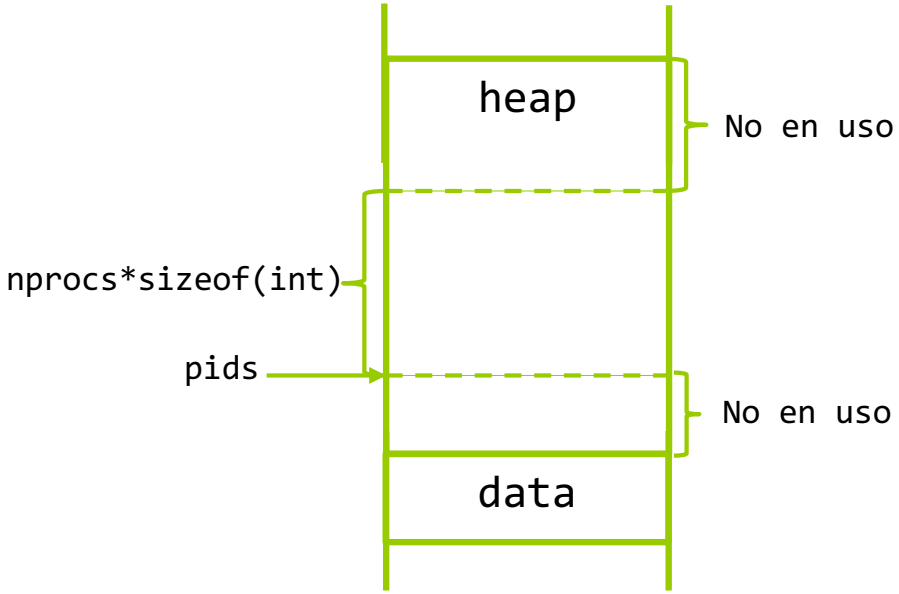
Con llamada a sistema

```
…
int nprocs=atoi(argv[1]);
int *pids;
pids=sbrk(nprocs*sizeof(int));
for(i=0;i<nprocs;i++){
   pids[i]=fork();
   if (pids[i]==0){
            …
   }
}
sbrk(-num_procs*sizeof(int));
```

Con la librería de C

```
…
int nprocs=atoi(argv[1]);
int *pids;
pids=malloc(nprocs*sizeof(int));
for(i=0;i<nprocs;i++){
   pids[i]=fork();
   if (pids[i]==0){
            …
   }
}
free(pids);
```



# Linux: memoria dinámica en el heap (V)

- Con sbrk: pids == **0x265fc000**

```
00400000-00401000 r--p 00000000 00:2d 10249278           /home/usuario1/test_sbrk
00401000-00499000 r-xp 00001000 00:2d 10249278           /home/usuario1/test_sbrk
00499000-004c2000 r--p 00099000 00:2d 10249278           /home/usuario1/test_sbrk
004c3000-004c7000 r--p 000c2000 00:2d 10249278           /home/usuario1/test_sbrk
004c7000-004ca000 rw-p 000c6000 00:2d 10249278           /home/usuario1/test_sbrk
004ca000-004d0000 rw-p 00000000 00:00 0
265da000-265fd000 rw-p 00000000 00:00 0                  [heap]
7ffd92c03000-7ffd92c24000 rw-p 00000000 00:00 0          [stack]
7ffd92c41000-7ffd92c45000 r--p 00000000 00:00 0          [vvar]
7ffd92c45000-7ffd92c47000 r-xp 00000000 00:00 0          [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0  [vsyscall]
```

- Con malloc: pids == **0x9ec7780**

```
00400000-00401000 r--p 00000000 00:2d 10249274           /home/usuario1/test_malloc
00401000-00499000 r-xp 00001000 00:2d 10249274           /home/usuario1/test_malloc
00499000-004c2000 r--p 00099000 00:2d 10249274           /home/usuario1/test_malloc
004c3000-004c7000 r--p 000c2000 00:2d 10249274           /home/usuario1/test_malloc
004c7000-004ca000 rw-p 000c6000 00:2d 10249274           /home/usuario1/test_malloc
004ca000-004d0000 rw-p 00000000 00:00 0
09ec6000-09ee8000 rw-p 00000000 00:00 0                  [heap]
7ffedb794000-7ffedb7b5000 rw-p 00000000 00:00 0          [stack]
7ffedb7cf000-7ffedb7d3000 r--p 00000000 00:00 0          [vvar]
7ffedb7d3000-7ffedb7d5000 r-xp 00000000 00:00 0          [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0 [vsyscall]
```

# Linux: memoria dinámica fuera del heap (I)

- **mmap**: llamada a sistema para pedir memoria en tiempo de ejecución fuera del heap

void * mmap(void *addr, int nbytes, int permisos, int flags, int fd, int offset)

- Se puede indicar o no la dirección inicial de la nueva región
  - Si *addr* == 0 el SO asigna la dirección inicial
  - Si *addr* != 0 se usa como pista de dirección inicial pero el SO la acaba decidiendo (si no se usa el flag MAP_FIXED)
- Se deben indicar permisos de acceso
  - PROT_EXEC, PROT_READ, PROT_WRITE, PROT_NONE



# Linux: memoria dinámica fuera del heap (II)

- Tipos de regiones creadas con mmap
  - Puede crear regiones compartibles o privadas→obligatorio especificar una y solo una de las dos opciones
    - Parámetro *flags* debe contener o MAP_SHARED o MAP_PRIVATE
  - Puede crear una región **sin inicializar** como malloc (memoria *anónima*)
    - Parámetro *fd* y *offset* se ignoran
    - Parámetro *flags* debe contener MAP_ANONYMOUS
    - Si MAP_SHARED la región es compartida solo con los procesos hijos creados después del mmap
    - Si MAP_PRIVATE la región se hereda pero cada uno trabaja con su copia
  - Puede crear una región **inicializada** con el contenido de un fichero
    - Parámetro *fd* debe representar a un fichero (tema 4)
    - Parámetro *offset* debe contener la posición inicial del fichero que se quiere mapear (puede ser 0 para mapear desde el inicio)
    - Si MAP_SHARED los cambios hechos en memoria se hacen también en el fichero
    - Si MAP_PRIVATE los cambios en memoria no se hacen en el fichero
- **munmap**: libera una región reservada con mmap

  int munmap(void *addr, int nbytes)



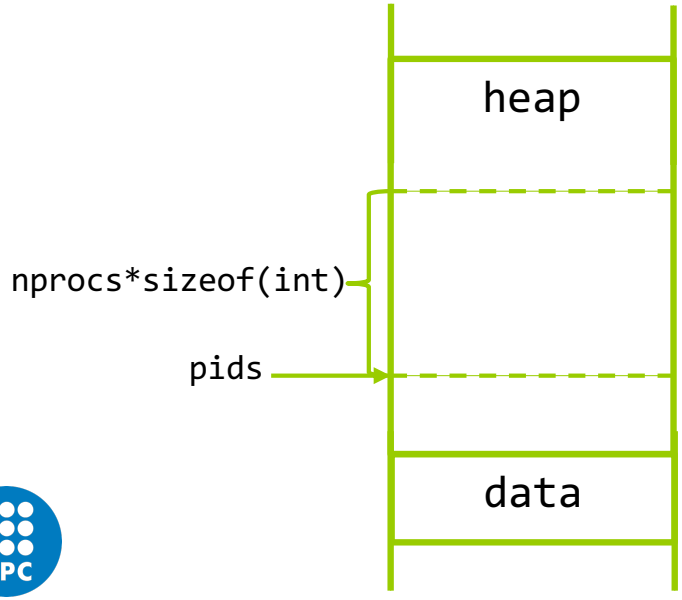
# Linux: memoria dinámica fuera del heap (III)

- Resumen de flags y sus implicaciones
  - MAP_FIXED → *addr* será la dirección inicial de la región. Si no se puede usar, mmap devuelve error
  - MAP_SHARED → No se puede añadir MAP_PRIVATE
  - MAP_PRIVATE → No se puede añadir MAP_SHARED
  - MAP_ANONYMOUS → Parámetros *fd* y *offset* se ignoran
    - Si MAP_SHARED → cambios compartidos con hijos que heredan la región
    - Si MAP_PRIVATE → cambios no compartidos con los hijos (heredan la región y trabajan con su copia)
  - Sin MAP_ANONYMOUS → *fd* y *offsets* deben ser válidos (tema 4)
    - Si MAP_SHARED → Cambios se hacen también en el fichero
    - Si MAP_PRIVATE → Cambios solo en memoria y desaparecen con el proceso

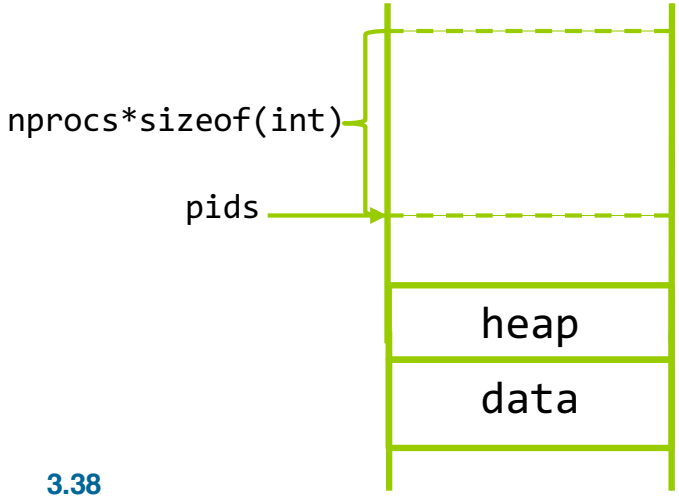
# Linux: memoria dinámica fuera del heap (IV)

- Ejemplo mmap memoria sin inicializar

```
…
int nprocs=atoi(argv[1]);
int *pids;
pids=malloc(nprocs*sizeof(int));
sprintf(buf, “Direccion %p\n”, pids);
write(1,buf,strlen(buf));
for(i=0;i<nprocs;i++){
   pids[i]=fork();
   if (pids[i]==0){
          …
   }
}
free(pids);
```




```
…
int nprocs=atoi(argv[1]);
int *pids;
pids=mmap(0,nprocs*sizeof(int),PROT_READ|PROT_WRITE,
          MAP_ANONYMOUS|MAP_PRIVATE,-1,0);
sprintf(buf, “Direccion %p\n”, pids);
write(1,buf,strlen(buf));
for(i=0;i<nprocs;i++){
   pids[i]=fork();
   if (pids[i]==0){
          …
   }
}
munmap(pids, nprocs*sizeof(int));
```



# Linux: memoria dinámica fuera del heap (V)

- Con malloc: pids == **0x9ec7780**

```
00400000-00401000 r--p 00000000 00:2d 10249274          /home/usuario1/test_malloc
00401000-00499000 r-xp 00001000 00:2d 10249274          /home/usuario1/test_malloc
00499000-004c2000 r--p 00099000 00:2d 10249274          /home/usuario1/test_malloc
004c3000-004c7000 r--p 000c2000 00:2d 10249274          /home/usuario1/test_malloc
004c7000-004ca000 rw-p 000c6000 00:2d 10249274          /home/usuario1/test_malloc
004ca000-004d0000 rw-p 00000000 00:00 0
09ec6000-09ee8000 rw-p 00000000 00:00 0                 [heap]
7ffedb794000-7ffedb7b5000 rw-p 00000000 00:00 0         [stack]
7ffedb7cf000-7ffedb7d3000 r--p 00000000 00:00 0         [vvar]
7ffedb7d3000-7ffedb7d5000 r-xp 00000000 00:00 0         [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0 [vsyscall]
```

- Con mmap: pids == **0x7f9201159000**

```
00400000-00401000 r--p 00000000 00:2d 10249273          /home/usuario1/test_mmap
00401000-00499000 r-xp 00001000 00:2d 10249273          /home/usuario1/test_mmap
00499000-004c2000 r--p 00099000 00:2d 10249273          /home/usuario1/test_mmap
004c3000-004c7000 r--p 000c2000 00:2d 10249273          /home/usuario1/test_mmap
004c7000-004ca000 rw-p 000c6000 00:2d 10249273          /home/usuario1/test_mmap
004ca000-004d0000 rw-p 00000000 00:00 0
234f7000-23519000 rw-p 00000000 00:00 0                 [heap]
7f9201159000-7f920115a000 rw-p 00000000 00:00 0
7ffc33a47000-7ffc33a68000 rw-p 00000000 00:00 0         [stack]
7ffc33bc7000-7ffc33bcb000 r--p 00000000 00:00 0         [vvar]
7ffc33bcb000-7ffc33bcd000 r-xp 00000000 00:00 0         [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0 [vsyscall]
```



# Linux: memoria dinámica fuera del heap (VI)

- Ejemplo mmap memoria inicializada con fichero

```
…
char *c_fichero;
char buf[80];
int fd = open("mifichero", O_RDONLY); // TEMA 4
c_fichero=mmap(0,1*sizeof(char),PROT_READ, MAP_PRIVATE, fd, 0);
sprintf(buf,"Dirección %p.El primer carácter de mifichero es %c\n",c_fichero, c_fichero[0]);
write(1,buf,strlen(buf));
…
```

- c_fichero == 0x7fd52018e000

```
00400000-00401000 r--p 00000000 00:2d 10249279                  /home/usuario1/test_mmap
00401000-00498000 r-xp 00001000 00:2d 10249279                  /home/usuario1/test_mmap
00498000-004c1000 r--p 00098000 00:2d 10249279                  /home/usuario1/test_mmap
004c1000-004c5000 r--p 000c0000 00:2d 10249279                  /home/usuario1/test_mmap
004c5000-004c8000 rw-p 000c4000 00:2d 10249279                  /home/usuario1/test_mmap
004c8000-004cd000 rw-p 00000000 00:00 0
3ea6d000-3ea8f000 rw-p 00000000 00:00 0                         [heap]
7fd52018e000-7fd52018f000 r--p 00000000 00:2d 10249284          /home/usuario1/mifichero
7ffedf04d000-7ffedf06e000 rw-p 00000000 00:00 0                 [stack]
7ffedf11d000-7ffedf121000 r–p 00000000 00:00 0                  [vvar]
7ffedf121000-7ffedf123000 r-xp 00000000 00:00 0                 [vdso]
Ffffffffff600000-ffffffffff601000 –xp 00000000 00:00 0          [vsyscall]
```

# Compartición de memoria (I)

- Por defecto
  - Ningún proceso puede acceder a la memoria de otro proceso
  - La memoria de un proceso es **accesible por todos los threads** (tema 5)
- Puede haber compartición (a nivel de página)
  - Implícita: transparente al programador, usada por el SO para implementar optimizaciones y solo de lectura
  - Explícita: pedida por el proceso usando llamadas a sistema (mecanismo de comunicación entre procesos)



# Linux: Compartición de memoria (I)

- Llamadas a sistema para compartición explícita:
  - mmap
  - shmget, shmat, shmctl,…
- Ejemplo mmap memoria sin inicializar y compartida (padres e hijos)

```
…
int ret;
int *valor;
char buf[80];
valor=mmap(0,sizeof(int),PROT_READ|PROT_WRITE,MAP_ANONYMOUS|MAP_SHARED,-1,0);
*valor = 100;
ret = fork();
if (ret==0){
    *valor = 42;
    exit(0);
}
waitpid(-1,NULL,0);
sprintf(buf, “Valor es: %d\n”, *valor);
write(1,buf,strlen(buf));
…
```



Carga bajo demanda

Librerías dinámicas

COW

Memoria virtual

# SERVICIOS PARA LA OPTIMIZACIÓN DEL USO DE MEMORIA



# Carga bajo demanda

- **Objetivo**: acelerar el proceso de carga y potencialmente reducir la memoria física usada
- Retrasa la carga hasta que ese contenido (código o datos) se referencia por primera vez
- Al poner en ejecución un programa se carga solo parcialmente
  - Asignar memoria física y copiar el contenido de esa parte del ejecutable
  - Actualizar MMU con las traducciones
- Mecanismo para detectar que algo no está cargado:
  - PCB del proceso contiene la descripción del espacio lógico del proceso (regiones y fuente de los datos)
  - MMU no tiene traducción válida para lo no cargado → genera excepción
  - SO gestiona la excepción
    - En el PCB comprueba que la dirección es válida pero que no está cargada
    - Completa entonces la carga: asigna memoria física, la rellena con el contenido y actualiza la información de la MMU
  - Al acabar la gestión se reanuda la instrucción que la ha provocado que esta vez se completará con éxito

# Librerías dinámicas (I)

- **Objetivo**:
  - Reducir el espacio ocupado en disco y el espacio ocupado en memoria
    - Pensad en cuántos programas utilizan la libC, cuánto espacio necesitamos en disco y en memoria si cada uno tiene una copia (idéntica) de la librería
  - Facilitar la actualización de las librerías
    - No hace falta recompilar los programas porque el código no forma parte del ejecutable, al ejecutar el programa se enlazará con la nueva versión
- Usando librerías dinámicas
  - Los ejecutables (en disco) no contienen el código de las librerías dinámicas, solo una referencia a las funciones que utiliza →Ahorra espacio en disco
  - Los procesos (en memoria) pueden compartir la zona en memoria que contiene el código (que es sólo de lectura) de las librerías comunes →Ahorra espacio en memoria y reduce el tiempo de carga
- Por defecto en Linux se utilizan librerías dinámicas
  - Con gcc hay que usar el flag –static para que se use la versión estática de las librerías del sistema y de lenguaje



# Librerías dinámicas (II)

- Mecanismo
  - El SO lanza el linker dinámico al inicio de la ejecución del programa
    - Mapea en el espacio de direcciones del proceso las librerías (mmap)
      - Sólo se cargarán una vez en memoria física y todos los procesos compartirán esa memoria (mmap con opción MAP_SHARED)
    - Cuando el proceso usa por primera vez una función de la librería, actualiza la referencia a esta función teniendo en cuenta las direcciones asignadas
- Memoria compartida implícita entre los procesos que usan las mismas librerías



# Librerías dinámicas (III)

```
…
int nprocs=atoi(argv[1]);
int *pids;
pids=mmap(0,nprocs*sizeof(int),PROT_READ|PROT_WRITE,MAP_ANONYMOUS|MAP_PRIVATE,-1,0);
sprintf(buf, “Direccion %p\n”, pids);
write(1,buf,strlen(buf));
for(i=0;i<10;i++){
   pids[i]=fork();
   if (pids[i]==0){
          …
   }
}
munmap(pids, nprocs*sizeof(int));
```

- Con compilación estática: gcc –static –o test_mmap test_mmap.c

```
00400000-00401000 r--p 00000000 00:2d 10249273            /home/usuario1/test_mmap
00401000-00499000 r-xp 00001000 00:2d 10249273            /home/usuario1/test_mmap
00499000-004c2000 r--p 00099000 00:2d 10249273            /home/usuario1/test_mmap
004c3000-004c7000 r--p 000c2000 00:2d 10249273            /home/usuario1/test_mmap
004c7000-004ca000 rw-p 000c6000 00:2d 10249273            /home/usuario1/test_mmap
004ca000-004d0000 rw-p 00000000 00:00 0
234f7000-23519000 rw-p 00000000 00:00 0                   [heap]
7f9201159000-7f920115a000 rw-p 00000000 00:00 0
7ffc33a47000-7ffc33a68000 rw-p 00000000 00:00 0           [stack]
7ffc33bc7000-7ffc33bcb000 r--p 00000000 00:00 0           [vvar]
7ffc33bcb000-7ffc33bcd000 r-xp 00000000 00:00 0           [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0   [vsyscall]
```

# Librerías dinámicas (IV)

- Con compilación dinámica: gcc –o test_mmap test_mmap.c

```
5612c0f12000-5612c0f13000 r--p 00000000 00:2d 10249281   /home/usuario1/test_mmap
5612c0f13000-5612c0f14000 r-xp 00001000 00:2d 10249281   /home/usuario1/test_mmap
5612c0f14000-5612c0f15000 r--p 00002000 00:2d 10249281   /home/usuario1/test_mmap
5612c0f15000-5612c0f16000 r--p 00002000 00:2d 10249281   /home/usuario1/test_mmap
5612c0f16000-5612c0f17000 rw-p 00003000 00:2d 10249281   /home/usuario1/test_mmap
7fd72a217000-7fd72a21a000 rw-p 00000000 00:00 0
7fd72a21a000-7fd72a242000 r--p 00000000 fd:00 2497177    /usr/lib/x86_64-linux-gnu/libc.so.6
7fd72a242000-7fd72a3d7000 r-xp 00028000 fd:00 2497177    /usr/lib/x86_64-linux-gnu/libc.so.6
7fd72a3d7000-7fd72a42f000 r--p 001bd000 fd:00 2497177    /usr/lib/x86_64-linux-gnu/libc.so.6
7fd72a42f000-7fd72a430000 ---p 00215000 fd:00 2497177    /usr/lib/x86_64-linux-gnu/libc.so.6
7fd72a430000-7fd72a434000 r--p 00215000 fd:00 2497177    /usr/lib/x86_64-linux-gnu/libc.so.6
7fd72a434000-7fd72a436000 rw-p 00219000 fd:00 2497177    /usr/lib/x86_64-linux-gnu/libc.so.6
7fd72a436000-7fd72a443000 rw-p 00000000 00:00 0
7fd72a467000-7fd72a469000 rw-p 00000000 00:00 0
7fd72a469000-7fd72a46b000 r--p 00000000 fd:00 2497020    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fd72a46b000-7fd72a495000 r-xp 00002000 fd:00 2497020    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fd72a495000-7fd72a4a0000 r--p 0002c000 fd:00 2497020    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fd72a4a0000-7fd72a4a1000 rw-p 00000000 00:00 0
7fd72a4a1000-7fd72a4a3000 r--p 00037000 fd:00 2497020    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fd72a4a3000-7fd72a4a5000 rw-p 00039000 fd:00 2497020    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fff7bfbb000-7fff7bfdc000 rw-p 00000000 00:00 0          [stack]
7fff7bfe1000-7fff7bfe5000 r--p 00000000 00:00 0          [vvar]
7fff7bfe5000-7fff7bfe7000 r-xp 00000000 00:00 0          [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0 [vsyscall]
```

# Copy on Write (I)

- **Objetivo**: acelerar la copia entre procesos y potencialmente reducir la memoria física usada
- Retrasa la reserva/inicialización de memoria física hasta que sea necesario
  - Si no se accede a una zona nueva → no necesitamos reservarla realmente
  - Si no modificamos una zona que es una copia → no necesitamos duplicarla
- En Linux se aplica
  - En el fork: padre e hijo comparten memoria física mientras no se modifica
  - Al pedir memoria dinámica: se asigna la misma página inicializada con ceros hasta que se escriba
- Memoria compartida implícita



# Copy on Write (II)

- En el momento que habría que hacer la copia:
  - En el **PCB** el SO marca la región destino con los **permisos de acceso reales**
  - En la **MMU** el SO marca la región destino y la región fuente con **permiso sólo de lectura**
  - En la MMU el SO asocia a la región destino las mismas direcciones físicas de la región fuente:
    - Misma traducción → memoria compartida
- Si un proceso intenta escribir en la región, la MMU genera excepción y SO la gestiona haciendo la copia real (asignar memoria física, copiar el contenido, actualizar MMU) y reiniciando el acceso



# Copy on Write (III)

- Proceso A ocupa:
  - Código: 3 páginas, Datos 2 páginas, Pila: 1 página, Heap: 1 página
- Si proceso A ejecuta fork, justo después del fork:
  - Total memoria física:
    - Sin COW: proceso A= 7 páginas + hijo A= 7 páginas= 14 páginas
    - Con COW: proceso A= 7 páginas + hijo A=0 páginas = 7 páginas
- Al cabo de un rato…depende de lo que hagan los procesos, por ejemplo:
  - Si hijo A muta (y el nuevo espacio del hijo ocupa 10 páginas):
    - Sin COW: proceso A= 7 páginas + hijo A= 10 páginas= 17 páginas
    - Con COW: proceso A= 7 páginas + hijo A=10 páginas = 17 páginas
  - Si hijo A no muta, depende de lo que haga, pero el código al menos puede ser compartido, suponiendo que el resto no lo sea:
    - Sin COW: proceso A= 7 páginas + hijo A= 7 páginas= 14 páginas
    - Con COW: proceso A= 7 páginas + hijo A=4 páginas = 11 páginas
- **En cualquier caso hay que ver qué páginas se modifican (por lo tanto no se pueden compartir) y qué páginas si se pueden compartir**



# Memoria Virtual (I)

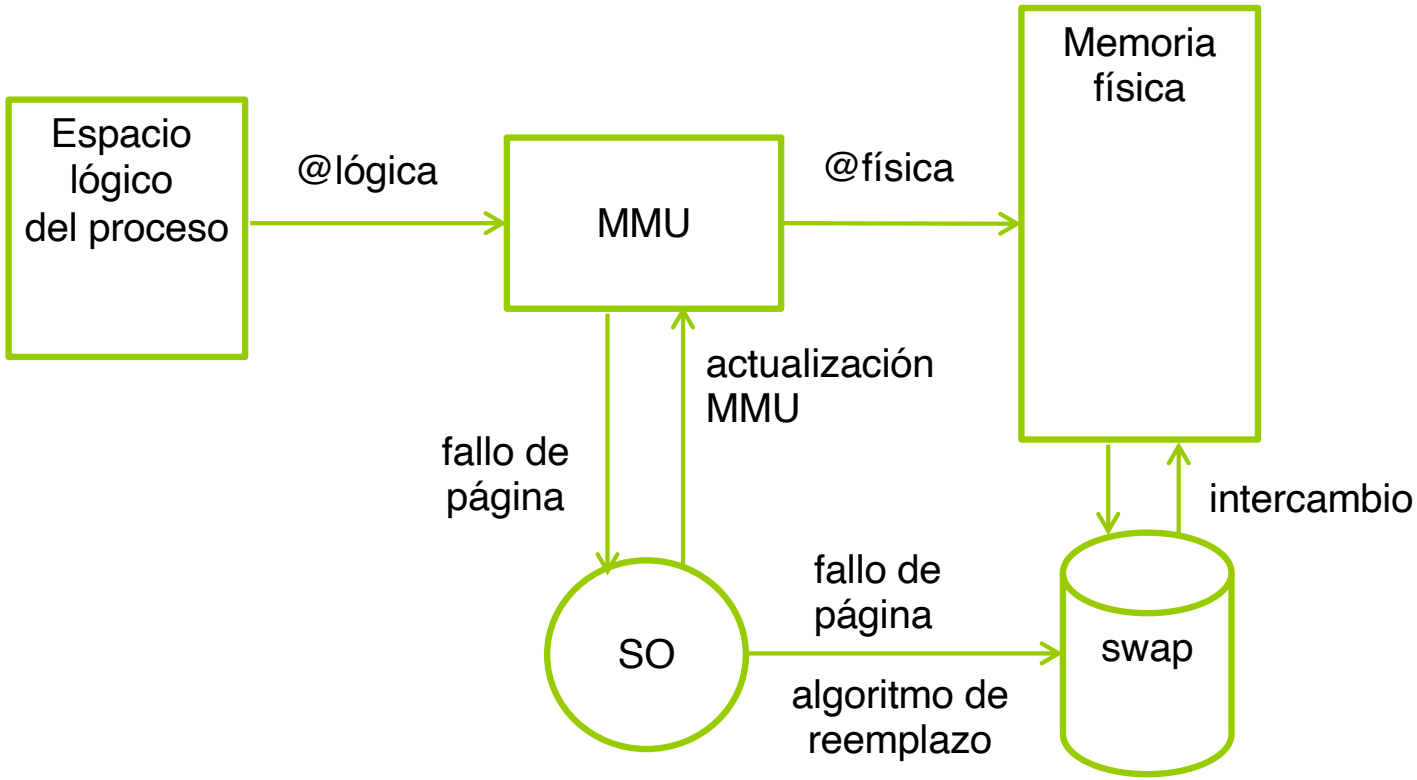
- **Objetivo:** Reducir la cantidad de memoria física asignada a un proceso en ejecución
  - Y así poder aumentar el grado de multiprogramación (cantidad de procesos cargados al mismo tiempo)
- Memoria virtual
  - Además de “traer” cosas a memoria bajo demanda, permite “sacar” cosas que ya no son necesarias
    - SO debe predecir qué no es necesario
      - Se guarda en el almacén secundario (*backing storage*) por si se equivoca en la predicción → **área de swap** (típicamente es una zona de disco)

# Memoria Virtual (II)

- Memoria virtual basada en paginación
  - **Espacio lógico de un proceso está distribuido entre memoria física** (páginas residentes) **y área de swap** (páginas no residentes)
- **Reemplazo de memoria**: cuando el SO necesita liberar marcos
  - Selecciona una página *víctima* y actualiza la MMU eliminando su traducción
    - ¿Cuándo se ejecuta? ¿Cómo se elige la víctima?
    - Guarda su contenido en el área de swap para que se pueda recuperar si es necesario
  - Asigna el marco ocupado a la página que se necesita en memoria
- Cuando se accede a una página guardada en el área de swap
  - MMU no puede hacer la traducción: genera excepción
    - **Fallo de página**
  - SO
    - Comprueba en las estructuras del proceso que el acceso es válido
    - Asigna un marco libre para la página (lanza el reemplazo de memoria si es necesario)
    - Localiza en el área de swap el contenido y lo escribe en el marco
    - Actualiza la MMU con la @física asignada

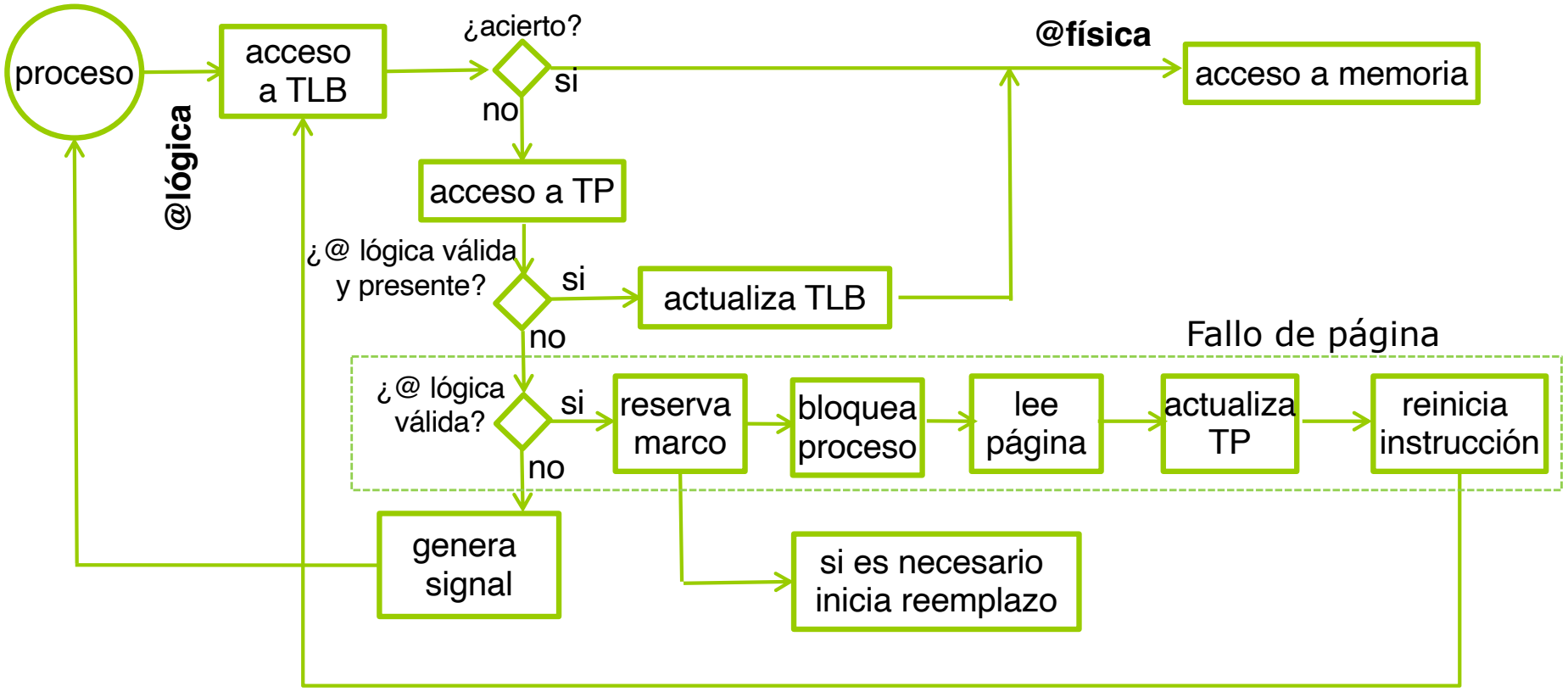


# Memoria Virtual (III)



# Memoria Virtual (IV)

- Pasos en el acceso a memoria

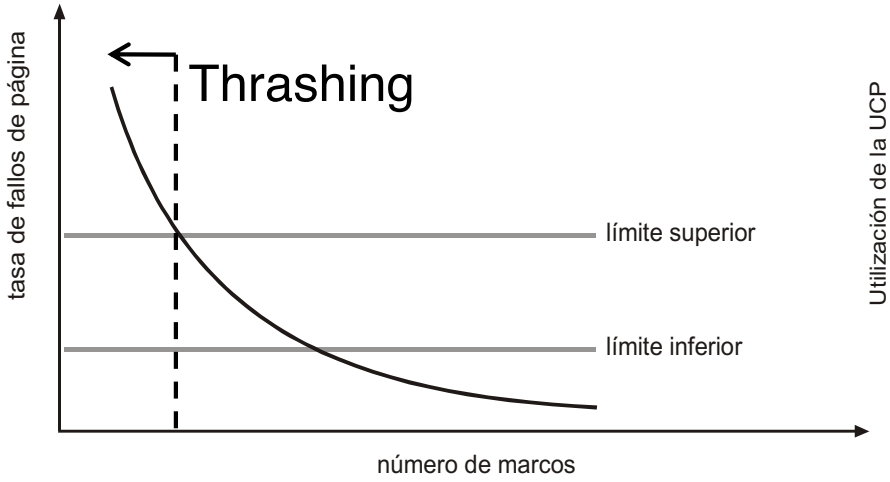


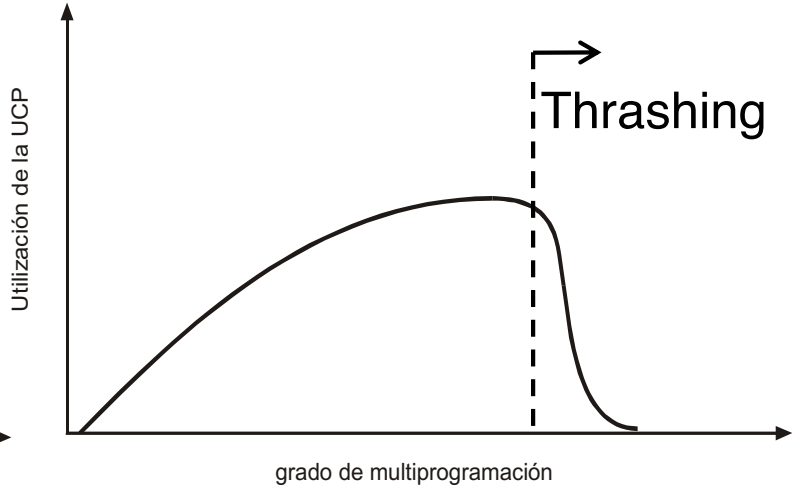
# Memoria Virtual (V)

- Efectos del uso de la memoria virtual
  - La suma de los espacios lógicos de los procesos en ejecución puede ser mayor que la cantidad de memoria física de la máquina
  - El espacio lógico de un proceso también puede ser mayor que la memoria física disponible
  - Acceder a una página no residente es más lento que acceder a una página residente
    - Excepción + carga de la página
    - Importante minimizar el número de fallos de página
      - Elegir bien las páginas víctimas
      - Aplicar *prefetch* de memoria: cargar anticipadamente las páginas que se van a referenciar

# Memoria Virtual (VI)

- Sobrepaginación (*thrashing*)
  - Proceso en thrashing
    - **Invierte más tiempo en el intercambio de memoria que avanzando su ejecución**
    - No consigue mantener simultáneamente en memoria el conjunto mínimo de páginas que necesita para avanzar
  - Se debe a que se ha sobrecargado el sistema de memoria
    - Detección: controlar tasa de fallos de página por proceso
    - Tratamiento: controlar el número de procesos que se permiten cargar y parar procesos temporalmente (swap out del proceso)
      - Los procesos expulsados están en estado swapped-out



# Resumen: Linux sobre Intel (I)

- Llamada a sistema **exec:** provoca la **carga** de un nuevo programa
  - Inicialización del PCB con la descripción del nuevo espacio de direcciones, asignación de memoria, …
- Creación de procesos (**fork**):
  - Inicialización del PCB con la descripción de su espacio de direcciones (copia del padre)
  - Se utiliza COW: hijo comparte marcos con el padre hasta que algún proceso los modifica
  - Creación e inicialización de la TP del nuevo proceso
    - Se guarda en su PCB la @ base de su TP
- Planificación de procesos
  - Un registro de control contiene la @base de la TP del proceso en RUN
  - En el **cambio de contexto** se actualiza este registro y se invalida la TLB
- Llamada a sistema **exit**:
  - Elimina la TP del proceso y libera los marcos que el proceso tenía asignados (si nadie más los estaba usando)



# Resumen: Linux sobre Intel (II)

- Memoria virtual basada en [segmentación]* paginada
  - Algoritmo de reemplazo: aproximación de LRU
  - Prefetch simple (secuencial)
- Implementa COW a nivel de página
- Carga bajo demanda
- Soporte para librerías compartidas

(*) segmentación por necesidad de la arquitectura pero Linux no lo explota

